



SAPIENZA
UNIVERSITÀ DI ROMA

Un tool per la ricerca di virtual patients in biomchemical network

Facoltà di Ingegneria dell'informazione, informatica e statistica
Laurea triennale in Informatica

Luca Sforza

Matricola 2050030

Relatore

Prof. Enrico Tronci

Anno Accademico 2024/2025

Un tool per la ricerca di virtual patients in biomchemical network
Relazione di Tirocinio. Sapienza Università di Roma

© 2025 Luca Sforza. Tutti i diritti riservati

Questa tesi è stata composta con L^AT_EX e la classe Sapthesis.

Email dell'autore: sforza.2050030@studenti.uniroma1.it

«Tutti i modelli sono sbagliati, ma alcuni sono utili.»
— *George E. P. Box*

Indice

Acronomi	vi
1 Introduzione	1
1.1 Contesto e Motivazioni	1
1.2 Contributi	1
1.3 Stato dell'arte	2
1.4 Struttura della Relazione di Tirocinio	3
2 Modellazione Biologica	4
2.1 Background	4
2.1.1 Cos'è un modello?	4
2.1.2 Sistemi Dinamici	4
2.2 modelli biologici di Pathway	5
3 Stima dei parametri	6
3.1 Vincoli sui parametri	6
3.2 Ordinamenti delle specie plausibili	9
4 Algoritmo genetico OpenAI	10
5 Progettazione del tool	12
5.1 Conversione da modello qualitativo a quantitativo	12
5.2 Ordinamento delle specie e plausibilità del modello	14
6 Esempi di modelli	17
6.1 Cellule staminali mammarie (R-HSA-9938206)	17
6.2 Ripiegamento proteico mediato da chaperoni nel cancro (R-HSA-390471)	19
6.3 Mutazione IDH1 e produzione di 2-idrossiglutarato (R-HSA-2978092)	20
6.4 Regolazione dell'apoptosi tramite ubiquitinazione proteica (R-HSA-169911)	20
6.5 Patologie legate alla risposta cellulare allo stress	23
7 Conclusione	25
7.1 Risultati congiunti degli esperimenti	25
Glossario	27
Ringraziamenti	28

Elenco delle figure

1.1	Rappresentazione schematica di un modello biologico. Le frecce sono le reazioni che collegano i reattanti con i prodotti delle reazioni. . . .	2
3.1	Visualizzazione del transitorio	7
4.1	Visualizzazione del gradient descend	11
5.1	Esempio di una possibile descrizione di un problema di ottimizzazione usando YAML.	13
5.2	UML	14
5.3	Esempio di modello SBML quantitativo generato dal tool. Si possono vedere i modelli SBML generati usati come esempi nella repository: Il codice sorgente del progetto è disponibile su GitHub: https://github.com/LucaSforza/apricopt-sbml2sim	15
6.1	Rappresentazione del modello: R-HSA-9938206	17
6.2	A sinistra l'ottimizzazione dei parametri per ogni ordinamento delle specie e a destra la simulazione di una possibile permutazione.. . . .	18
6.3	Rappresentazione del modello: R-HSA-390471	19
6.4	A sinistra c'è l'ottimizzazione dei parametri, una che ha successo e un'altra che non ha successo; a destra invece la simulazione di una possibile permutazione.	19
6.5	Rappresentazione del modello: R-HSA-2978092	20
6.6	A sinistra c'è l'ottimizzazione fatta da Nevergrad dei parametri e a destra invece la simulazione di una possibile permutazione.	21
6.7	Rappresentazione del modello: R-HSA-169911	21
6.8	A sinistra c'è l'ottimizzazione fatta da Nevergrad dei parametri e a destra invece la simulazione di una possibile permutazione.	22
6.9	Rappresentazione del modello: R-HSA-9675132	23
6.10	A sinistra c'è l'ottimizzazione dei parametri, una che ha successo e un'altra che non ha successo; a destra invece la simulazione di una possibile permutazione.	23

Elenco delle tabelle

6.1	Tabella qualitativa e quantitativa del modello R-HSA-9938206. . . .	17
6.2	Tabella degli esperimenti del modello R-HSA-9938206. In questo caso sono state provate tutte le possibili permutazioni delle specie. Dato che per ogni permutazione si è raggiunto il minimo globale della funzione allora ogni permutazione delle specie è plausibile.	18
6.3	Tabella qualitativa e quantitativa del modello R-HSA-390471.	20
6.4	Tabella degli esperimenti del modello R-HSA-390471.	20
6.5	Tabella qualitativa e quantitativa del modello R-HSA-390471.	21
6.6	Tabella degli esperimenti del modello R-HSA-2978092.	21
6.7	Tabella qualitativa e quantitativa del modello R-HSA-169911.	22
6.8	Tabella degli esperimenti del modello R-HSA-169911.	22
6.9	Tabella qualitativa e quantitativa del modello R-HSA-9675132. . . .	24
6.10	Tabella degli esperimenti del modello R-HSA-9675132.	24
7.1	Tabella qualitativa e quantitativa del modello R-HSA-9675132. . . .	25
7.2	Tabella degli esperimenti del modello R-HSA-9675132.	26

Capitolo 1

Introduzione

1.1 Contesto e Motivazioni

Tutti gli esseri viventi sono costituiti da cellule, piccole fabbriche biochimiche che attraverso dei metabolismi consumano energia per esistere. Studiare le cellule e il loro funzionamento è fondamentale per comprendere malattie nel corpo umano, come il cancro.

Una cellula nella sua interezza è un sistema biologico governato da reti complesse di interazioni tra entità chimiche come proteine, metaboliti e acidi nucleici. Tuttavia si possono studiare solo piccoli parti di questi network metabolici, ovvero i pathway che vanno a studiare una specifica funzione di una cellula.

Un modello biologico è formato da molecole e reazioni come mostrato in figura 1.1. Esistono online database di pathway che possono essere resi simulabili modellando le reazioni come equazioni differenziali ordinarie (ODE).

Per questo tirocinio è stato utilizzato reactome⁽²⁾. L'accuratezza di queste simulazioni dipende in modo cruciale dalle costanti cinetiche che governano le reazioni. Tali parametri risultano spesso difficili o impossibili da misurare sperimentalmente, lasciando un ampio margine di incertezza nei modelli.

Però esiste il concetto di **paziente virtuale** inteso come una collezione di modelli computazionali che rappresentano diversi possibili scenari biologici compatibili con le conoscenze disponibili.

1.2 Contributi

Con questa relazione di tirocinio, si vuole presentare un software che, dato un modello biologico, permette di individuare pazienti virtuali plausibili.

I modelli biologici che vogliamo rappresentare sono normalizzati, ovvero la concentrazione delle specie (ovvero un insieme indistinguibile di entità chimiche) sono tutte comprese tra 0 e 1. Se la concentrazione di una specie è vicina al valore 1 vuol dire che è molto espressa, se invece è vicino a 0 allora è poco espressa.

Dato un ordinamento del valore delle concentrazioni normalizzato delle specie si vogliono trovare i parametri tale per cui l'ordinamento delle specie normalizzato esiste. Ovvero se esiste un paziente virtuale tale che renda plausibile l'ordinamento e confrontare pazienti virtuali per capire il più plausibile.

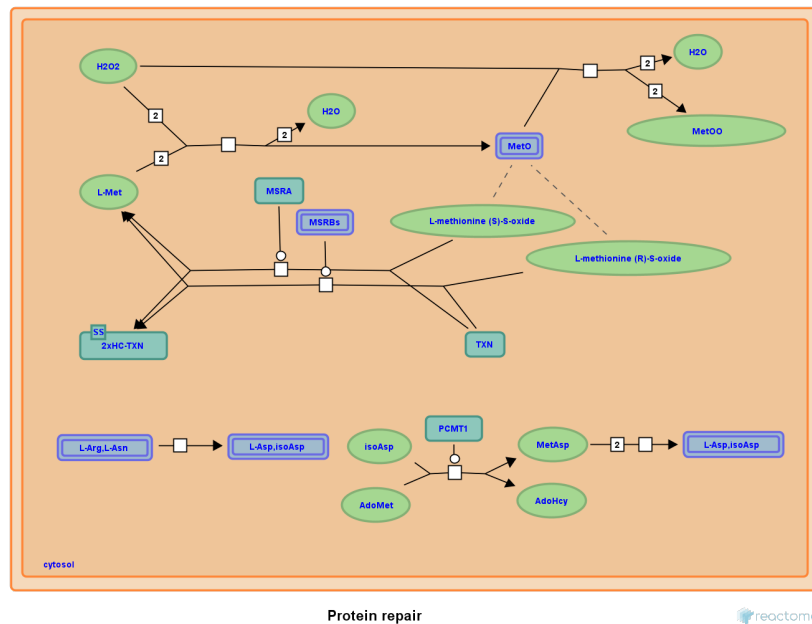


Figura 1.1 Rappresentazione schematica di un modello biologico. Le frecce sono le reazioni che collegano i reattanti con i prodotti delle reazioni.

In particolare, il lavoro introduce un tool per l'esplorazione degli spazi di parametri in reti biochimiche. I contributi principali sono:

1. Generazione di modelli biologici partendo da database esistenti, di cui è stato progetto di Verifica e Validazione dei Sistemi intelligenti e ampliato per gli obbiettivi il tirocinio.
2. Ricerca tramite ottimizzazione black box di pazienti virtuali che soddisfano certi vincoli, di cui è stato progetto per l'esame di Intelligenza Artificiale e ampliato per gli obbiettivi il tirocinio.
3. Implementazione di un algoritmo genetico di OpenAI originariamente usato per il reinforment learning e riadattato per l'ottimizzazione black box.
4. Simulazione dei modelli biologici in ambienti normalizzati e ricerca di pazienti virtuali: dato un ordinamento normalizzato delle concentrazioni delle specie, trovare i parametri che rendono quell'ordinamento realizzabile (ovvero individuare pazienti virtuali compatibili) e confrontarli per valutarne la plausibilità.

1.3 Stato dell'arte

L'approccio di usare equazioni differenziali per modellare sistemi biochimici è una pratica molto diffusa nella letteratura. La dinamica delle reti biochimiche viene descritta tramite equazioni differenziali ordinarie, che permettono di rispettare i vincoli quantitativi e temporali delle reazioni chimiche.

Invece un approccio alternativo alla simulazione dei sistemi biologici con equazioni differenziali è quello basato sui network booleani^{(3) (5) (7)}. In questo caso, le reazioni biochimiche sono semplificate: ogni proteina può essere soltanto attiva o inattiva e le interazioni sono ridotte a regole logiche. Spesso si assume che tutte le reazioni avvengano alla stessa velocità e che i nodi della rete possano cambiare stato in maniera sincrona.

1.4 Struttura della Relazione di Tirocinio

- Capitolo 2 presenta il background teorico sulla modellazione di reti biochimiche.
- Capitolo 3 descrive la formulazione del problema e i vincoli usati per definire le assegnazioni di parametri ammissibili.
- Capitolo 4 Illustra l'algoritmo di OpenAI come uno dei possibili per trovare i parametri
- Capitolo 5 Introduce la progettazione del software e le dipendenze utilizzate per crearlo.
- Capitolo 6 si presentano degli utilizzi su 5 modelli biologici presi da Reactome.
- Capitolo 7 considerazioni finali

Capitolo 2

Modellazione Biologica

2.1 Background

2.1.1 Cos'è un modello?

2.1.2 Sistemi Dinamici

I modelli biologici che consideriamo sono sistemi dinamici tempo-invarianti, generalmente non lineari, descritti da equazioni differenziali ordinarie (ODE).

Lo stato del sistema al tempo t è un vettore $x(t) = (x_1(t), \dots, x_n(t))^T \in [0, 1]^n \subseteq \mathbb{R}^n$, dove $x_i(t)$ rappresenta la concentrazione normalizzata della i -esima specie.

In forma compatta le ODE si scrivono

$$\frac{dx}{dt} = f(t, x).$$

Sia m il numero di reazioni nel modello.

Per ogni specie x_i definiamo $R_{x_i} \subseteq \{1, \dots, m\}$ (reazioni in cui x_i è reattante), $P_{x_i} \subseteq \{1, \dots, m\}$ (reazioni che producono x_i),

n_j^i è la stechiometria della specie i nella reazione j .

Se $R_{x_i} \neq \emptyset$ e $P_{x_i} \neq \emptyset$, l'evoluzione temporale di x_i è data da:

$$\frac{dx_i}{dt} = \sum_{j \in P_{x_i}} n_j^i v_j(t) - \sum_{j \in R_{x_i}} n_j^i v_j(t),$$

dove $v_j(t)$ è la velocità della reazione j .

La velocità di una reazione è data dalla legge dell'azione di massa (mass action)⁽⁸⁾. Questa legge afferma che di una reazione chimica è proporzionale al prodotto delle concentrazioni dei reattanti, ciascuna elevata al proprio coefficiente stechiometrico.

La velocità di una reazione è influenzata anche da modificatori (enzimi, inibitori). Per modellare l'effetto dei modificatori si usa la hill function. Quando la concentrazione di un modificatore raggiunge una certa soglia allora la reazione avviene. Questo perché molte reazioni in natura non avvengono se non catalizzate. Per ogni specie modificatrice x_ℓ introduciamo una funzione di Hill generica

$$H_\ell(t) = \begin{cases} \frac{x_\ell(t)^{10}}{K_\ell + x_\ell(t)^{10}}, & \text{se } x_\ell \text{ agisce da attivatore,} \\ \frac{K_\ell}{K_\ell + x_\ell(t)^{10}}, & \text{se } x_\ell \text{ agisce da inibitore,} \end{cases}$$

dove $K_\ell > 0$ è la costante di mezzo-saturazione e dato che è ignota viene aggiunta come parametro del modello.

Per la reazione $i \in \{1, \dots, m\}$ definiamo inoltre:

$S_i = \{j \in \{1, \dots, n\} \mid x_j \text{ è reattante della reazione } i\}$,

$M_i = \{j \in \{1, \dots, n\} \mid x_j \text{ è modificatore della reazione } i\}$.

n_i^j è la stechiometria della specie j nella reazione i .

La velocità $v_i(t)$ si esprime quindi come

$$v_i(t) = k_i \prod_{j \in S_i} x_j(t)^{n_i^j} \prod_{j \in M_i} H_j(t),$$

dove $k_i > 0$ è la costante cinetica della reazione. Essa essendo sconosciuta viene aggiunta come parametro del modello.

2.2 modelli biologici di Pathway

Un pathway metabolico (o via metabolica) è una sequenza di reazioni biochimiche che collega specie chiave.

Un pathway è formato da specie di input che non vengono prodotte da reazioni, ma vengono immesse nel pathway. Specie intermedie che sono sia prodotte che consumate da reazioni, specie di output che sono solo prodotte da reazioni e mai consumate e specie che non vengono né prodotte né consumate da alcuna reazione, ma sono enzimi o inibitori che influiscono nelle altre reazioni.

Un esempio tipico di input è l'ATP che funge da "energia" per molte reazioni chimiche.

Esse non vengono prodotte da reazioni ($P_{x_i} = \emptyset$) ma vengono immesse nel sistema; si modellano con un termine di ingresso costante $k_{\text{in},i}$:

$$\frac{dx_i}{dt} = k_{\text{in},i} - \sum_{j \in R_{x_i}} v_j(t).$$

I pathway hanno degli output che sono specie che vengono prodotte ma non consumate ($R_{x_i} = \emptyset$); ad esse si aggiunge una reazione di degradazione, che avrà una costante cinetica e ha come unico reattante la concentrazione dell'output stesso, quindi seguendo la mass action rule l'equazione differenziale per gli output è come segue:

$$\frac{dx_i}{dt} = \sum_{j \in P_{x_i}} v_j(t) - k_{\text{out},i} x_i(t),$$

Nei modelli biologici di pathway possono essere presenti specie che non partecipano né come reattanti né come prodotti alle reazioni, ma solo come modificatori. Quindi queste specie devono avere una costante di immissione ed anche una di degradazione.

Queste specie sono enzimi o inibitori esterni, quindi influiscono sulla velocità delle altre reazioni del modello. La loro dinamica è descritta come segue:

$$\frac{dx_i}{dt} = k_{\text{in},i} - k_{\text{out},i} x_i(t).$$

Capitolo 3

Stima dei parametri

Le costanti cinetiche e gli altri parametri (es. K_ℓ , $k_{\text{in},i}$, $k_{\text{out},i}$) sono generalmente sconosciuti e devono essere stimati. Poiché i parametri cinetici possono variare di molti ordini di grandezza, è comune parametrizzarli in scala logaritmica. Se p è il numero totale di parametri, si può usare un vettore di parametri $\theta \in [-20, 20]^p \subseteq \mathbb{Z}^p$.

Da notare che il dominio dei parametri è non solo discreto, ma addirittura finito. Questo ci permetterà di cercare tutti i parametri che rispettano determinati vincoli.

Per ricavare la costante cinetica a partire dal parametro logaritmico si usa la parametrizzazione in base dieci:

$$k_i = 10^{\theta_i}, \quad \theta_i \in [-20, 20],$$

così θ_i rappresenta $\log_{10} k_i$ e si garantisce $k_i > 0$.

Prima di stimare i parametri bisogna aggiungere dei vincoli su di essi.

Si può fare in due modi: o limitando il dominio delle costanti cinetiche eliminando quelle implausibili oppure definendo una funzione che dati i parametri valutano l'errore all'interno del modello, poi tramite un ottimizzatore black box si trovano i parametri che minimizzano l'errore.

Per l'esame di **intelligenza artificiale** su cui si basa il progetto del tirocinio è stato mostrato quale algoritmo dell'ottimizzatore Nevergrad converge più velocemente su un modello particolare e semplificato.

3.1 Vincoli sui parametri

I parametri da scegliere devono soddisfare determinati requisiti.

Primo tra tutti i parametri devono rendere il sistema stabile.

Quindi bisogna notare il fatto che gli enzimi o inibitori di una reazione controllano il comportamento di quella reazione.

Quindi una condizione necessaria, ma non sufficiente per la stabilità è che le costanti cinetiche che producono modificatori (enzimi o inibitori) devono essere maggiori delle costanti cinetiche che i modificatori alterano.

Formalmente sia:

$$R = \{(i, j) \in [1, m]^2 \mid \text{la reazione } i \text{ produce un modificatore della reazione } j\}$$

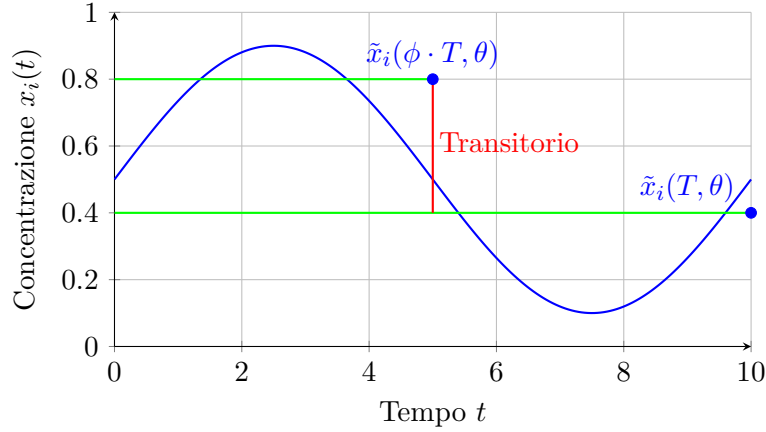


Figura 3.1 Visualizzazione del transitorio

Allora l'ottimizzatore deve rispettare questi vincoli che limitano lo spazio feaseble dei parametri:

$$k_i \geq k_j \mid (i, j) \in R \quad (3.1)$$

Per vincolare definitivamente la stabilità bisogna eliminare il transitorio. Il transitorio è la parte iniziale del sistema in cui i valori non sono ancora stabili.

Per capire cos'è il transitorio dobbiamo definire cos'è la concentrazione media di una specie.

Definiamo T come l'orizzonte di simulazione.

La concentrazione di una specie non dipende solo dal tempo, ma anche dai parametri, quindi la concentrazione di una specie è: $x_i(t, \theta)$.

La concentrazione media di una specie i è definita come segue:

$$\tilde{x}_i(t', \theta) = \mathbb{E}_{\tau \sim \mathcal{U}(0, t')} [x_i(\tau, \theta)] = \frac{1}{t'} \int_0^{t'} x_i(\tau, \theta) d\tau, \quad t' > 0. \quad (3.2)$$

Il transitorio quindi è il segmento tra i due valori medi $\tilde{x}_i(\phi \cdot T, \theta)$ e $\tilde{x}_i(T, \theta)$.

ϕ è un iper-parametro dell'ottimizzatore, per lo scopo del tirocinio è stato scelto $\phi = \frac{1}{2}$.

Per vincolare l'eliminazione del transitorio definiamo una funzione da minimizzare:

$$\mathcal{L}_1(\theta) = \sum_{i=1}^n (\tilde{x}_i(\phi \cdot T, \theta) - \tilde{x}_i(T, \theta))^2 \quad (3.3)$$

Minimizzare $\mathcal{L}_1$ significa eliminare il transitorio e quindi vincolare la stabilità del sistema.

Per poter visualizzare che cos'è il transitorio in figura 3.1 è presente un esempio.

Un altro vincolo sulle costanti cinetiche è che devono far rimanere il valore delle concentrazioni tra 0 e 1, poiché per ipotesi la concentrazioni delle specie sono tutte normalizzate.

Per fare ciò dobbiamo aggiungere delle nuove variabili allo stato per ogni specie esistente, con la relativa equazione differenziale:

z_i è la nuova variabile dello stato associato alla specie x_i .

$$\frac{dz_i}{dt} = \text{if } x_i(t, \theta) \geq 1 \vee x_i(t, \theta) \leq 0 \text{ then } 10 \text{ else } 0 \quad (3.4)$$

Quindi per vincolare le specie tra 0 e 1 dobbiamo minimizzare il valore di queste variabili.

$$\mathcal{L}_2(\theta) = \sum_{i=1}^n z_i(T, \theta) \quad (3.5)$$

I parametri delle costanti cinetiche non solo devono garantire la stabilità, ma devono anche garantire varie regole biologiche. Per esempio l'acqua è il solvente della vita, tutto ciò che è presente dentro una cellula è immerso nell'acqua. L'acqua quindi è sempre la specie più espressa all'interno di un modello biologico.

L'acqua è solo un esempio, ma esistono molti motivi del perché si vuole vincolare che una certa specie è più espressa di un'altra.

Definiamo quindi una relazione di ordinamento sulle specie $R \subseteq [1, n]^2$.

Se $(i, j) \in R$ allora la specie j deve essere più espressa della specie i .

Definiamo una nuova funzione da minimizzare $\mathcal{L}_3$.

$$\mathcal{L}_3(\theta) = \sum_{(i,j) \in R} \max(0, \tilde{x}_i(T, \theta) - \tilde{x}_j(T, \theta)) \quad (3.6)$$

Altri vincoli da aggiungere sono vincolare il valore medio di una concentrazione ad una certa costante nota.

Supponiamo di avere un dataset:

$$\mathcal{D} = \{(i, y) \in [1, n] \times [0, 1] \mid y \text{ è la concentrazione media osservata della specie } i\}.$$

Un modo per vincolare i parametri a questi dati è massimizzando la **likelihood**⁽¹⁾ dei parametri.

La likelihood è la probabilità condizionata di osservare dal modello il dato osservato se il modello è corretto.

Più formalmente:

$$L(\theta \mid \mathcal{D}) = \mathbb{P}(\mathcal{D} \mid \theta) \quad (3.7)$$

Quindi i parametri da scegliere devono essere quelli che massimizzano la likelihood.

$$\hat{\theta}(\mathcal{D}) = \arg \max_{\theta} L(\theta \mid \mathcal{D}) \quad (3.8)$$

Ma in pratica per ottenere questi parametri dobbiamo fare delle ipotesi.

I dati osservati $\mathcal{D}$ per ipotesi devono provenire da repliche sperimentali indipendenti ottenute nelle stesse condizioni sperimentali del modello e affette da rumore osservazionale. Tipicamente si assume un errore additivo gaussiano sulle medie osservate:

$$y_i = \tilde{x}_i(T, \theta) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma_i^2),$$

dove σ_i^2 è la varianza dell'errore per la specie i . Sotto questa ipotesi la log-likelihood del dataset è

$$\log L(\theta \mid \mathcal{D}) = -\frac{1}{2} \sum_{(i,y) \in \mathcal{D}} \left[\frac{(y - \tilde{x}_i(T, \theta))^2}{\sigma_i^2} + \log(2\pi\sigma_i^2) \right].$$

La stima dei parametri si ottiene massimizzando questa funzione (o minimizzando la somma dei residui normalizzati).

Quindi definiamo una nuova funzione da minimizzare $\mathcal{L}_4$.

$$\mathcal{L}_4(\theta) = \sum_{(i,y) \in \mathcal{D}} (y_i - \tilde{x}_i(T, \theta))^2 \quad (3.9)$$

Come ultima funzione da definire è una per gestire gli errori numerici. Se la scelta delle costanti cinetiche è troppo sbagliata le specie potrebbero molto velocemente divergere a $+\infty$.

Perciò definiamo una funzione che da $+\infty$ nel caso la simulazione è terminata con errori numerici e 0 altrimenti.

$$\mathcal{L}_5(\theta) = \begin{cases} +\infty & \text{se la simulazione è terminata con errori numerici} \\ 0 & \text{altrimenti} \end{cases} \quad (3.10)$$

La funzione finale da minimizzare è:

$$\mathcal{L}(\theta) = \alpha \cdot \mathcal{L}_1(\theta) + \beta \cdot \mathcal{L}_2(\theta) + \gamma \cdot \mathcal{L}_3(\theta) + \zeta \cdot \mathcal{L}_4(\theta) + \mathcal{L}_5(\theta) \quad (3.11)$$

Dove $\alpha, \beta, \gamma, \zeta \in \mathbb{R}$ sono degli iper-parametri dell'ottimizzatore per scalare le varie funzioni, a seconda se vogliamo vincolare di più l'aderenza ai dati sperimentali (γ o ζ) oppure la stabilità.

3.2 Ordinamenti delle specie plausibili

Dato che stiamo lavorando in un ambiente normalizzato allora possiamo cercare di capire se un certo ordinamento delle specie è plausibile o meno.

Un ordinamento lo possiamo vedere come una permutazione sulle specie $\pi : [1, n] \rightarrow [1, n]$

$$\mathcal{D} = \left\{ \left(i, \frac{\pi(i)}{n+1} \right) \mid i \in [1, n] \right\}, \quad (3.12)$$

Vincoliamo le concentrazioni delle specie con valori normalizzati tra 0 e 1.

Per capire quale ordinamento sia più plausibile di un altro possiamo confrontare l'ottimizzazione della funzione rispetto a due permutazioni diverse.

Capitolo 4

Algoritmo genetico OpenAI

Il tool progettato permette di eseguire l'ottimizzazione della funzione obiettivo $\mathcal{L}$ tramite diversi ottimizzatori come Nevergrad e Nomad. Può essere facilmente esteso per includere diversi algoritmi.

In questa relazione di tirocinio si propone l'implementazione di un algoritmo genetico di OpenAI⁽⁶⁾ per risolvere problemi blackbox.

Un algoritmo genetico è una procedura di ricerca euristica ispirata all'evoluzione naturale osservabile in natura.

Ad ogni iterazione dell'algoritmo (generazione) un insieme di parametri (genotipi) che vengono perturbati (mutazione genetica) e la funzione obiettivo viene valutata su queste perturbazioni (fitness).

Le perturbazioni con la fitness migliori vengono ricombinate per formare i nuovi parametri per la nuova generazione.

In questo algoritmo di OpenAI la popolazione viene rappresentata come una distribuzione gaussiana multivariata che ha media sui parametri.

Quindi si vogliono trovare i parametri che massimizzano:

$$\mathbb{E}_{\varepsilon \sim \mathcal{N}(0, I)} \{ \mathcal{L}(\theta + \sigma \varepsilon) \} \quad (4.1)$$

Possiamo ottimizzare i parametri θ usando un gradient descend stocastico, vedere la figura 4.1 per una visualizzazione grafica del funzionamento.

$$\nabla_{\theta} \mathbb{E}_{\varepsilon \sim \mathcal{N}(0, I)} \{ \mathcal{L}(\theta + \sigma \varepsilon) \} = -\frac{1}{\sigma} \mathbb{E}_{\varepsilon \sim \mathcal{N}(0, I)} \{ \mathcal{L}(\theta + \sigma \varepsilon) \varepsilon \} \quad (4.2)$$

Esso può essere stimato campionando n valori $\varepsilon_1, \dots, \varepsilon_n \sim \mathcal{N}(0, I)$.

$$-\frac{1}{\sigma} \mathbb{E}_{\varepsilon \sim \mathcal{N}(0, I)} \{ \mathcal{L}(\theta + \sigma \varepsilon) \varepsilon \} \approx -\frac{1}{\sigma n} \sum_{i=1}^n \mathcal{L}(\theta + \sigma \varepsilon_i) \varepsilon_i \quad (4.3)$$

Ad ogni generazione i nuovi parametri si spostano in direzione del gradiente.

Poiché il gradiente può essere molto lungo esso va scalato per un iper-parametro dell'algoritmo $\alpha > 0$ chiamato **learning rate**.

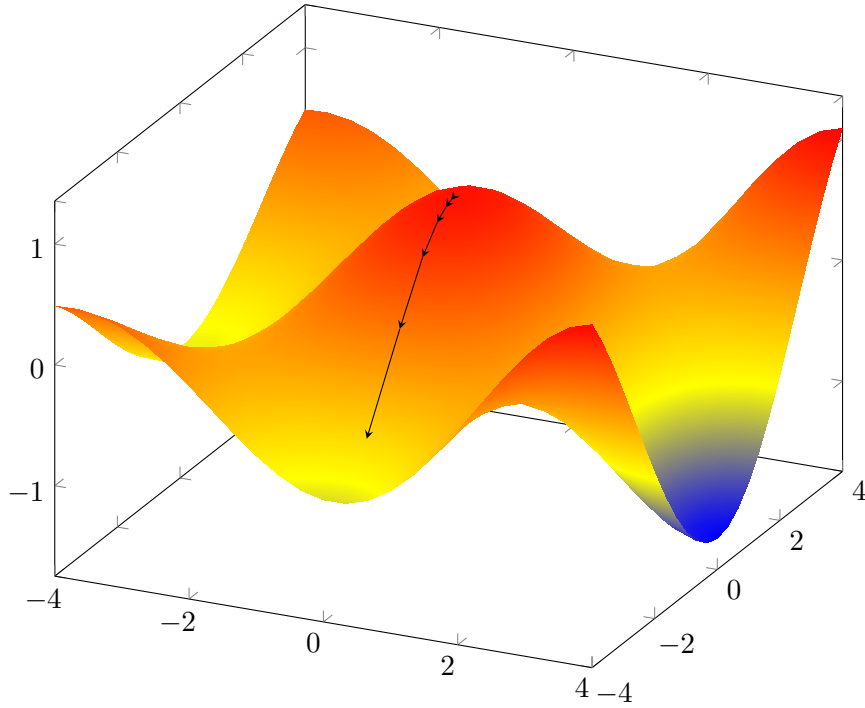


Figura 4.1 Visualizzazione del gradient descend

Algorithm 1 OpenAI Algoritmo Evolutivo Genetico

```

1: Input: Learning rate  $\alpha$ , deviazione standard  $\sigma$ , parametri iniziali:  $\theta_0$ 
2: for  $t = 0, 1, 2, \dots$  do
3:   Campiona  $\varepsilon_1, \dots, \varepsilon_n \sim \mathcal{N}(0, I)$ 
4:   Calcola funzione di loss  $\mathcal{L}_i = \mathcal{L}(\theta_t + \sigma \varepsilon_i)$  for  $i \in [1, n]$ 
5:    $\theta_{t+1} \leftarrow \theta_t - \alpha \frac{1}{\sigma n} \sum_{i=1}^n \mathcal{L}_i \varepsilon_i$ 
6: end for

```

Capitolo 5

Progettazione del tool

In Figura 5.2 è mostrato il diagramma UML che evidenzia le componenti centrali del tool che si appoggia al framework apricopt.

Apricopt ha delle interfacce che permettono di modellare problemi di ottimizzazione BlackBox, in particolare quei problemi che richiedono la simulazione di un modello biologico.

Al centro del framework è il BlackBoxSolver, che ha il compito di cercare i parametri ottimali.

Esso è un interfaccia, una classe può essere estesa ad essa per implementare un algoritmo di ottimizzazione, per esempio per quanto riguarda l'algoritmo evolutivo di OpenAI. Oppure si possono usare ottimizzatori terzi come Nevergrad o Nomad.

I problemi black box che stiamo considerando sono la stima dei parametri di un dato modello biologico.

La loss function $\mathcal{L}$ può essere passata attraverso SimulableBlackBox che simula il modello con i parametri θ e calcola $\mathcal{L}(\theta)$ sulla base della traiettoria $\mathcal{T}(\theta)$.

Per calcolare la traiettoria è necessario poter simulare il modello. Per fare ciò apricopt ha diversi simulatori utilizzabili.

Per gli esempi mostrati in questa relazione di tirocinio è stato utilizzato roadrunner⁽⁴⁾. Dal'UML in figura si può vedere come il simulatore è una classe astratta. Oltre al simulatore è necessario implementare l'istanza di un modello roadrunner.

Si può specificare anche un valore B tale per cui se vengono trovati dei parametri tale che $\mathcal{L}(\theta) \leq B$ allora i parametri sono sufficientemente buoni.

Nell'algoritmo 2 si può vedere l'implementazione di NevergradSolver.

Per la classe OpenAISolver viene sovrascritto il metodo solve con l'algoritmo 1 dove i suoi iperparametri vengono presi dai SolverParameters.

Un problema di ottimizzazione con questo tool può essere descritto usando YAML rendendo facile generare automaticamente questi problemi.

Un esempio è mostrato in figura 5.1.

Il codice sorgente del progetto è disponibile su GitHub:

<https://github.com/LucaSforza/apricopt-sbml2sim>.

5.1 Conversione da modello qualitativo a quantitativo

Oltre a ciò il tool può prendere un modello da Reactome⁽²⁾ e renderlo simulabile.

Algorithm 2 Metodo solve sovrascritto dalla classe Nevergrad solver

```

1: Input: problem: BlackBox, params: SolverParameters
2: optimizer  $\leftarrow$  algoritmo presente in Nevergrad preso dai params
3:  $B \leftarrow$  minimo preso dai parametri params
4: while  $i \in 1..*$  do
5:    $\theta \leftarrow$  optimizer.ask()
6:    $val \leftarrow$  problem.evaluate( $\theta$ )
7:   if  $val \leq B$  then return  $\theta$ 
8:   end if
9:   optimizer.tell( $\theta, val$ )
10: end while

```

```

files:
  model: ./converted_R-HSA-169911.sbml
  parameters: ./parameters.tsv
  objective: ./objective.tsv
  constrains: ./constrains.tsv
  valori: ./vals.tsv
horizon: 10000
absolute_tolerance: 1.0e-12
relative_tolerance: 1.0e-06
time_step: 10

loss_functions:
  stability:
    scale: 10000
  transitorial:
    scale: 10
  value:
    scale: 1

simulator: RoadRunner
random_seed: 104
solver: NOMAD
solver_parameters:
  - SEED 104
  - MAX_ITERATIONS 100000
  - NB_THREADS_PARALLEL_EVAL 1
  - MAX_EVAL 1500000

```

Figura 5.1 Esempio di una possibile descrizione di un problema di ottimizzazione usando YAML.


```
<?xml version="1.0" encoding="UTF-8"?>
<sbml xmlns="http://www.sbml.org/sbml/level2/version4" level="2" version="4">
  <model id="minimal_degradation" name="Minimal degradation model">
    <listOfCompartments>
      <compartment id="cell" constant="true"/>
    </listOfCompartments>

    <listOfSpecies>
      <species id="S" compartment="cell" initialConcentration="1" hasOnlySubstanceUnits="true"/>
    </listOfSpecies>

    <listOfParameters>
      <parameter id="k_deg" value="0.1" constant="true"/>
    </listOfParameters>

    <listOfReactions>
      <reaction id="deg_S" reversible="false" fast="false">
        <listOfReactants>
          <speciesReference species="S" stoichiometry="1" constant="true"/>
        </listOfReactants>
        <kineticLaw>
          <math xmlns="http://www.w3.org/1998/Math/MathML">
            <apply><times/><ci>k_deg</ci><ci>S</ci></apply>
          </math>
        </kineticLaw>
      </reaction>
    </listOfReactions>
  </model>
</sbml>
```

Figura 5.3 Esempio di modello SBML quantitativo generato dal tool. Si possono vedere i modelli SBML generati usati come esempi nella repository: Il codice sorgente del progetto è disponibile su GitHub: <https://github.com/LucaSforza/apricopt-sbml2sim>.

Oppure si possono generare tutti gli ordinamenti possibili e cercare i parametri per ognuno dei suoi ordinamenti così da trovare l'ordinamento più plausibile.

Capitolo 6

Esempi di modelli

6.1 Cellule staminali mammarie (R-HSA-9938206)

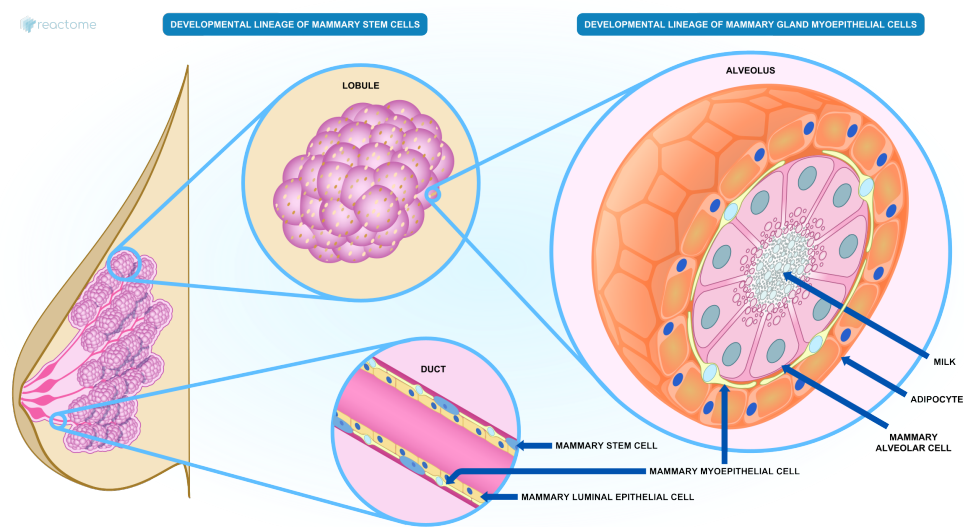


Figura 6.1 Rappresentazione del modello: R-HSA-9938206

Questo modello rappresenta lo sviluppo delle cellule staminali mammarie. La stima dei parametri del modello è stata fatta su ogni possibile permutazione delle specie e si è visto che per ogni permutazione l’ottimizzatore riesce a trovare il minimo globale della funzione obiettivo $\mathcal{L}$, quindi ogni permutazione riesce a trovare un paziente virtuale che giustifica il fenomeno.

Tabella qualitativa del modello			
Id modello	Numero specie	Numero reazioni	numero parametri
R-HSA-9938206	3	5	6

Tabella 6.1 Tabella qualitativa e quantitativa del modello R-HSA-9938206.

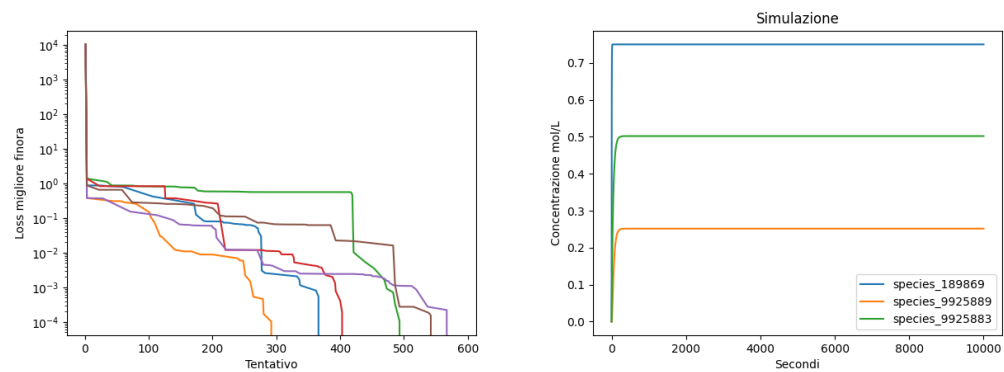


Figura 6.2 A sinistra l’ottimizzazione dei parametri per ogni ordinamento delle specie e a destra la simulazione di una possibile permutazione..

Tabella degli esperimenti					
Id esperimento	Ottimizzatore	Valore loss	Target loss	Tempo impiegato	RAM utilizzata
1	Nomad	0.0	0.0	5	6
2	Nomad	0.0	0.0	5	6
4	Nomad	0.0	0.0	5	6
3	Nomad	0.0	0.0	5	6
5	Nomad	0.0	0.0	5	6
6	Nomad	0.0	0.0	5	6
7	Nevergrad	0.0	0.0	5	6
8	Nevergrad	0.0	0.0	5	6
9	Nevergrad	0.0	0.0	5	6
10	Nevergrad	0.0	0.0	5	6
11	Nevergrad	0.0	0.0	5	6
12	Nevergrad	0.0	0.0	5	6
13	OpenAI	0.0	0.0	5	6
14	OpenAI	0.0	0.0	5	6
15	OpenAI	0.0	0.0	5	6
16	OpenAI	0.0	0.0	5	6
17	OpenAI	0.0	0.0	5	6
18	OpenAI	0.0	0.0	5	6

Tabella 6.2 Tabella degli esperimenti del modello R-HSA-9938206. In questo caso sono state provate tutte le possibili permutazioni delle specie. Dato che per ogni permutazione si è raggiunto il minimo globale della funzione allora ogni permutazione delle specie è plausibile.

Si possono vedere i risultati in 6.2.

6.2 Ripiegamento proteico mediato da chaperoni nel cancro (R-HSA-390471)

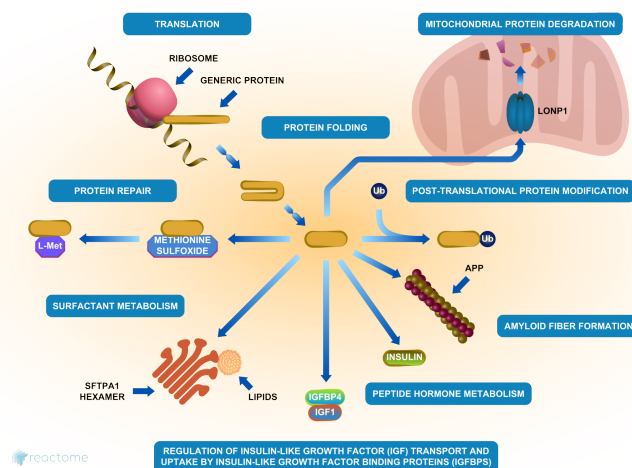


Figura 6.3 Rappresentazione del modello: R-HSA-390471

Per il modello 6.3 sono stati provati ogni possibile permutazione dell'ordinamento delle specie, ma non per ogni ordinamento sono state trovate delle costanti cinetiche. L'ordinamento che minimizza la loss è il modello che spiega maggiormente il fenomeno. Vengono riportate due ottimizzazioni, una dove non è stato trovato un paziente virtuale e quella che maggiormente spiega il fenomeno.

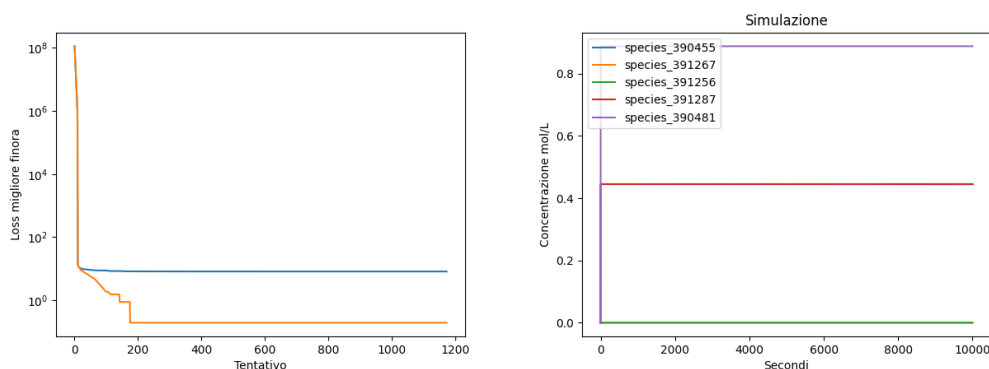


Figura 6.4 A sinistra c'è l'ottimizzazione dei parametri, una che ha successo e un'altra che non ha successo; a destra invece la simulazione di una possibile permutazione.

Tabella qualitativa del modello			
Id modello	Numero specie	Numero reazioni	numero parametri
R-HSA-390471	5	7	7

Tabella 6.3 Tabella qualitativa e quantitativa del modello R-HSA-390471.

Tabella degli esperimenti					
Id esperi- mento	Ottimizzatore	Valore loss	Target loss	Tempo im- piegato	RAM utiliz- zata

Tabella 6.4 Tabella degli esperimenti del modello R-HSA-390471.

6.3 Mutazione IDH1 e produzione di 2-idrossiglutarato (R-HSA-2978092)

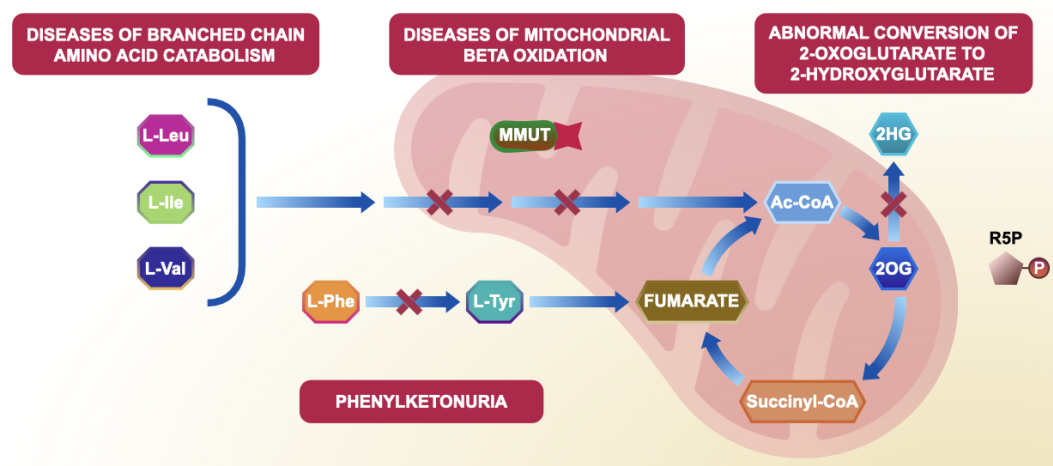


Figura 6.5 Rappresentazione del modello: R-HSA-2978092

La mutazione dell'enzima IDH1 nei glioblastomi altera la sua attività: invece di convertire isocitrato in α -chetoglutarato, usa NADPH per trasformare α -chetoglutarato in 2-idrossiglutarato. Questo metabolita anomalo si accumula nelle cellule e può favorirne la crescita tumorale.

6.4 Regolazione dell'apoptosi tramite ubiquitinazione proteica (R-HSA-169911)

L'equilibrio tra sopravvivenza e morte cellulare programmata è cruciale per lo sviluppo e la salute dei tessuti. L'ubiquitinazione e la degradazione delle proteine sono meccanismi chiave che controllano l'attivazione o l'inibizione dell'apoptosi (morte cellulare programmata).

6.4 Regolazione dell'apoptosi tramite ubiquitinazione proteica (R-HSA-169912)

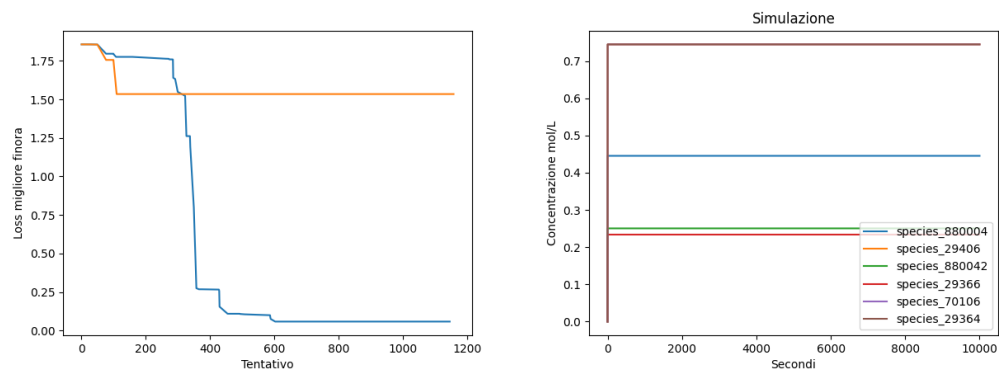


Figura 6.6 A sinistra c'è l'ottimizzazione fatta da Nevergrad dei parametri e a destra invece la simulazione di una possibile permutazione.

Tabella qualitativa del modello			
Id modello	Numero specie	Numero reazioni	numero parametri
R-HSA-2978092	5	8	9

Tabella 6.5 Tabella qualitativa e quantitativa del modello R-HSA-390471.

Tabella degli esperimenti					
Id esperimento	Ottimizzatore	Valore loss	Target loss	Tempo impiegato	RAM utilizzata

Tabella 6.6 Tabella degli esperimenti del modello R-HSA-2978092.

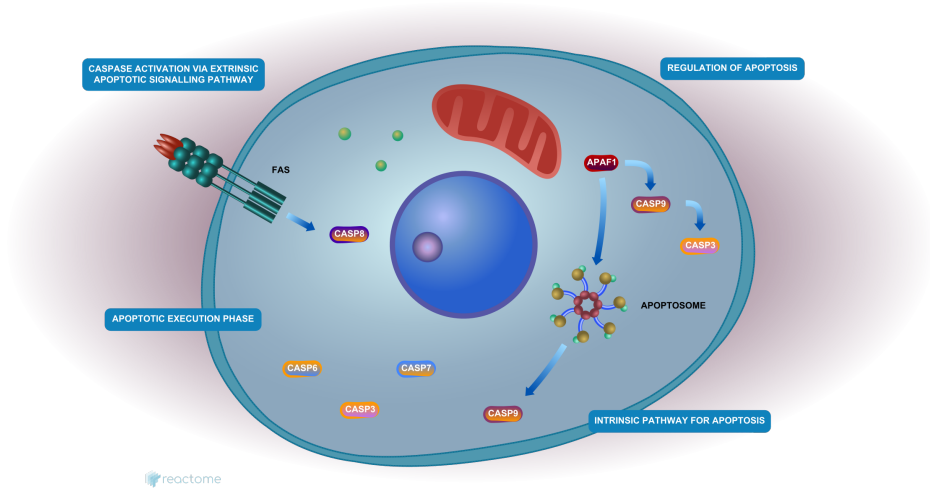


Figura 6.7 Rappresentazione del modello: R-HSA-169911

6.4 Regolazione dell'apoptosi tramite ubiquitinazione proteica (R-HSA-169912)

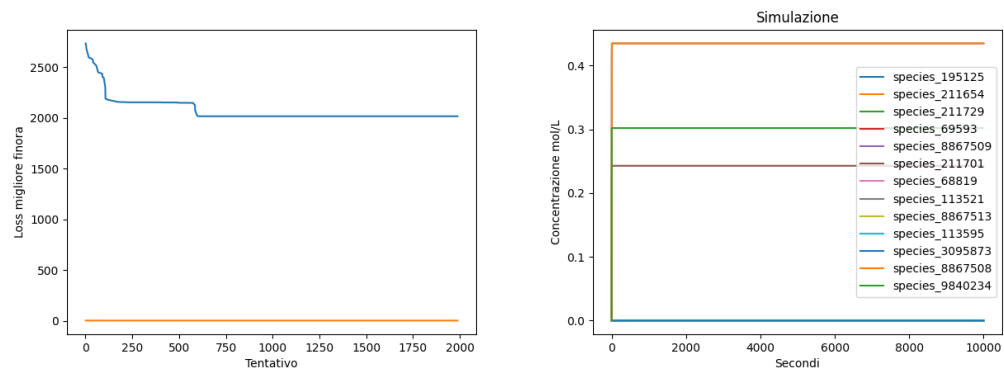


Figura 6.8 A sinistra c'è l'ottimizzazione fatta da Nevergrad dei parametri e a destra invece la simulazione di una possibile permutazione.

Tabella qualitativa del modello			
Id modello	Numero specie	Numero reazioni	numero parametri
R-HSA-169911	13	18	21

Tabella 6.7 Tabella qualitativa e quantitativa del modello R-HSA-169911.

Tabella degli esperimenti					
Id esperimento	Ottimizzatore	Valore loss	Target loss	Tempo impiegato	RAM utilizzata

Tabella 6.8 Tabella degli esperimenti del modello R-HSA-169911.

6.5 Patologie legate alla risposta cellulare allo stress

Le cellule devono adattarsi a stress come danni al DNA, carenza di nutrienti o ossigeno e radicali liberi. Quando la risposta è alterata, si sviluppano malattie: la senescenza cellulare può favorire il cancro o contribuire a patologie croniche e legate all'età.

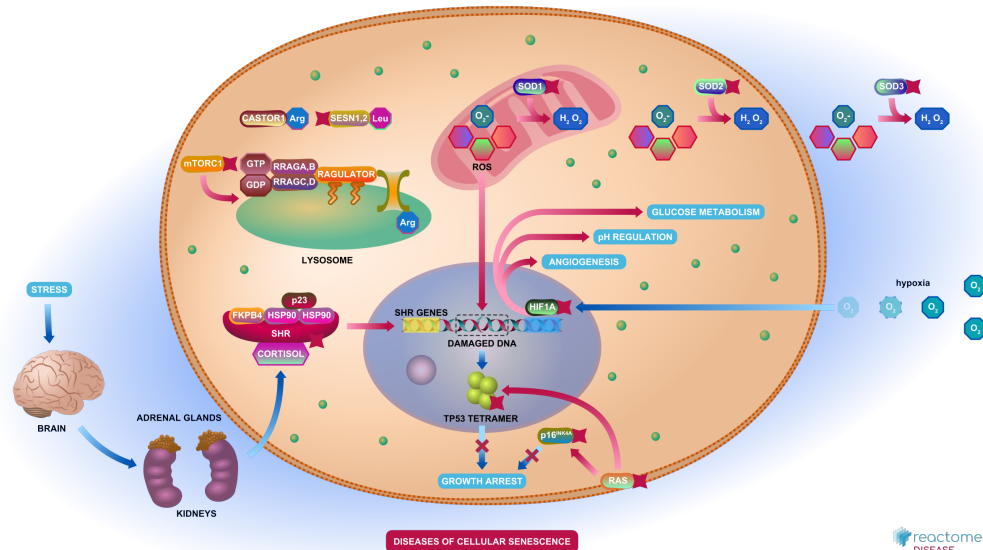


Figura 6.9 Rappresentazione del modello: R-HSA-9675132

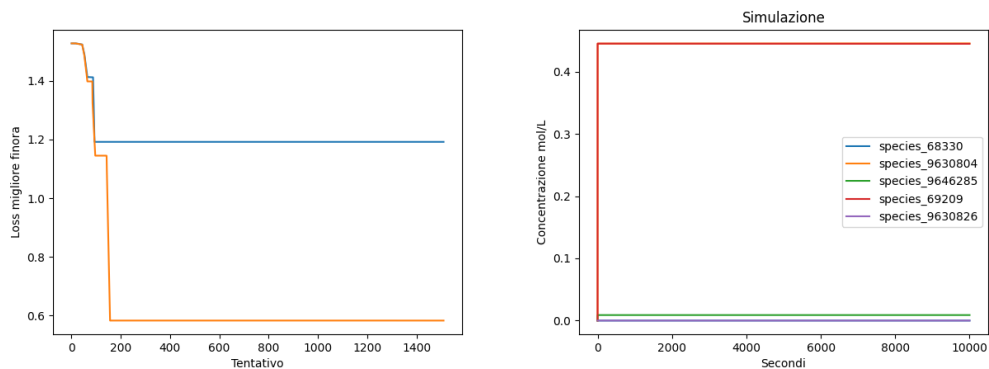


Figura 6.10 A sinistra c'è l'ottimizzazione dei parametri, una che ha successo e un'altra che non ha successo; a destra invece la simulazione di una possibile permutazione.

Per questo modello sono state provate in parallelo tutte le possibili combinazioni degli ordinamenti delle specie. È stato scelto come permutazione che spiega meglio il fenomeno quella che ha la loss migliore.

Tabella qualitativa del modello			
Id modello	Numero specie	Numero reazioni	numero parametri
R-HSA-9675132	5	8	8

Tabella 6.9 Tabella qualitativa e quantitativa del modello R-HSA-9675132.

Tabella degli esperimenti					
Id esperimento	Ottimizzatore	Valore loss	Target loss	Tempo impiegato	RAM utilizzata

Tabella 6.10 Tabella degli esperimenti del modello R-HSA-9675132.

Capitolo 7

Conclusione

Il lavoro ha introdotto un tool per l'individuazione di pazienti virtuali in reti biochimiche, fondato su tre elementi principali: la conversione automatica di modelli qualitativi (Reactome) in modelli quantitativi SBML, la formulazione del problema di stima parametrica come ottimizzazione vincolata black-box, e l'integrazione di diversi algoritmi evolutivi e numerici (Nevergrad, Nomad, OpenAI Evolution Strategies).

I risultati sperimentali hanno dimostrato che il sistema è in grado di:

1. garantire la stabilità dei modelli tramite vincoli espliciti su parametri e concentrazioni,
2. valutare la plausibilità di ordinamenti normalizzati delle specie,
3. individuare configurazioni parametriche compatibili con i vincoli biologici.

La validazione su pathway eterogenei ha evidenziato la capacità del framework di discriminare ordinamenti non realizzabili e di generare configurazioni coerenti con fenomeni biologici complessi.

7.1 Risultati congiunti degli esperimenti

Tabella qualitativa del modello			
Id modello	Numero specie	Numero reazioni	numero parametri
R-HSA-9675132	5	8	8

Tabella 7.1 Tabella qualitativa e quantitativa del modello R-HSA-9675132.

Tabella degli esperimenti						
Id modello	Id esperimento	Ottimizzatore	Valore loss	Target loss	Tempo impiegato	RAM utilizzata

Tabella 7.2 Tabella degli esperimenti del modello R-HSA-9675132.

Ringraziamenti

Bibliografia

- [1] Matthew J. Simpson and Ruth E. Baker. Parameter identifiability, parameter estimation and model prediction for differential equation models. *arXiv preprint arXiv:2405.08177*, 2025. Accessed: 2025-09-2.
- [2] M Milacic, D Beavers, P Conley, C Gong, M Gillespie, J Griss, R Haw, B Jassal, L Matthews, B May, R Petryszak, E Ragueneau, K Rothfels, C Sevilla, V Shumovsky, R Stephan, K Tiwari, T Varusai, J Weiser, A Wright, G Wu, L Stein, H Hermjakob, and P D'Eustachio. The reactome pathway knowledgebase 2024. *Nucleic Acids Research*, 2024.
- [3] Domenico Sgariglia, Flavia Raquel Gonçalves Carneiro, Luis Alfredo Vidal de Carvalho, Carlos Eduardo Pedreira, Nicolas Carels, and Fabricio Alves Barbosa da Silva. Optimizing therapeutic targets for breast cancer using boolean network models. *Computational Biology and Chemistry*, 2024.
- [4] Ciaran Welsh, Jin Xu, Lucian Smith, Matthias König, Kiri Choi, and Herbert M. Sauro. libroadrunner 2.0: a high performance sbml simulation and analysis library. *Bioinformatics*, 39(1):btac770, 2023.
- [5] Domenico Sgariglia, Alessandra Jordano Conforte, Carlos Eduardo Pedreira, Luis Alfredo Vidal de Carvalho, Flavia Raquel Gonçalves Carneiro, Nicolas Carels, and Fabricio Alves Barbosa da Silva. Data-driven modeling of breast cancer tumors using boolean networks. *Computational Biology and Chemistry*, 2021.
- [6] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.
- [7] Domenico Sgariglia, Alessandra Jordano Conforte, Carlos Eduardo Pedreira, Luis Alfredo Vidal de Carvalho, Flavia Raquel Gonçalves Carneiro, Nicolas Carels, and Fabricio Alves Barbosa da Silva. Boolean network model for cancer pathways: Predicting carcinogenesis and targeted therapy outcomes. *Computational Biology and Chemistry*, 2013.
- [8] Edda Klipp, Wolfram Liebermeister, Christoph Wierling, Axel Kowald, Ralf Herwig, and Hans Lehrach. *Systems Biology: A Textbook*. Wiley-Blackwell, Weinheim, 2 edition, 2009.
- [9] Michael Hucka, Andrew Finney, Herbert M. Sauro, Hamid Bolouri, John C. Doyle, Hiroaki Kitano, and SBML Forum. The systems biology markup language

(sbml): A medium for representation and exchange of biochemical network models. *Bioinformatics*, 19(4):524–531, 2003.